

Audit complet de l'intégration Harvest

Architecture BFF, authentification, permissions, contrats d'écriture, preuves live, sécurité et demande à Harvest.

255 endpoints classés	75 handlers BFF audités	117 tests unitaires réussis	0 ressource de test restante
---	---	---	--

Conclusion principale

Les credentials actuels ne sont pas globalement en lecture seule. Plusieurs écritures membre sont fonctionnelles. Les lacunes restantes subsistent après correction des défauts BFF : une capacité explicitement désactivée et des contrats/prérequis qui doivent être confirmés par Harvest.

Date de campagne : 28 juillet 2026

Compte membre : Anthlogan, compte de test non administrateur

Version auditée : BFF Parigo après corrections documentaires

Documentation de référence : snapshot officiel Harvest extrait le 26 mai 2026 et documentation publique consultée pendant la campagne

1. Résumé exécutif

Le socle public Harvest fonctionne : authentification de service, région, guest token, catalogue, recherche, autocomplete, référentiels, playlists éditoriales et assets répondent correctement. L'authentification membre, le profil, les favoris, les recherches sauvegardées, les tags et les historiques en lecture sont également opérationnels.

La clé actuelle n'est donc pas simplement « en lecture seule ». Plusieurs mutations membre persistent réellement : favori track, recherche sauvegardée, profil, image, tag et relation tag-track. En revanche, les capacités d'écriture sont hétérogènes.

Les défauts attribuables à Parigo ont été corrigés avant rédaction de ce rapport :

- payloads playlist alignés sur les champs documentés ;
- copie de playlist avec **CopyTracks: true** ;
- historique reconstruit depuis **HistoryItems[].DeliveryDate** et **UTCOffset** ;
- dates sans offset normalisées indépendamment du fuseau du serveur ou du navigateur ;
- code Harvest **3** exposé comme refus d'accès, et non comme déconnexion ;
- codes Harvest utiles conservés dans **upstreamCode** ;
- faux succès HTTP 200 remplacés par une relecture jusqu'à 30 secondes ;
- recherche par intention et résolution des taxonomies déplacées dans le BFF ;
- conversion shortlist → playlist rendue transactionnelle ;
- upload de photo relayé par le BFF, sans URL présignée exposée au navigateur ;
- lecture tolérante à **Tags[].tracks**, forme live différente de **Tags[].Tracks** documenté.

Après ces corrections, les écarts encore reproductibles concernent principalement les playlists membre, les suggestions, les notes, l'abonnement, le reset, le téléchargement, le cue sheet et le partage. Ils sont détaillés avec les réponses reçues et les questions minimales à Harvest.

Décision d'architecture : conserver le BFF. Il protège les secrets et tokens Harvest, stabilise les réponses hétérogènes et vérifie les mutations. Il ne doit pas recréer de logique catalogue ou métier qu'Harvest pourrait exposer directement.

2. Périmètre et méthode de preuve

Couverture statique

Élément	Couverture
Endpoints de la documentation classés	255
Public API	186
Import API	36
Export API	17
Agent API	16
Fichiers de routes BFF inspectés	51
Handlers BFF cartographiés	75
Lignes de parcours dans la matrice runtime	82

Management, CMS, Import, Export et Agent API ont été inventoriés mais non exécutés : aucun credential dédié n'est disponible et ces familles ne sont pas nécessaires au périmètre membre actuel.

Chaîne contrôlée

```
clic UI
→ route BFF
→ payload Harvest
→ HTTP + Error.Code Harvest
→ réponse BFF
→ relecture Harvest
→ relecture BFF
→ page Account
→ reload / renouvellement de session
→ nettoyage
```

Chaque mutation réversible a été relue à **0 ms, 250 ms, 1 s, 3 s, 10 s et 30 s**. Un HTTP 200 n'est pas assimilé à une persistance.

Tests navigateur

- Chromium desktop **1440×900** et mobile **390×844** ;
- Search, boutons de track, shortlist anonyme, connexion et Account ;
- huit espaces Account ouverts avec une session réelle ;
- fuseaux navigateur UTC, Europe/Paris et Australia/Sydney ;
- profile field modifié puis restauré ;

- photo neutre uploadée, retirée, puis photo originale restaurée ;
- favoris et tags créés, vérifiés et nettoyés ;
- aucune requête de suppression de compte.

Effets externes

- un e-mail de reset tenté : Harvest a répondu **code: "Failed"** ; aucun e-mail reçu ;
- un partage tenté : blocage avant envoi sur **getsharemusicurl, Error.Code=2** ;
- un message de contact envoyé par Resend : HTTP 201 et accusé reçu ;
- aucun téléchargement consommé : la validation officielle est refusée avant **getmusicdownload**.

Tous les tokens, cookies, mots de passe, URLs présignées et secrets sont exclus des artefacts.

3. Endpoints et parcours fonctionnels

Le détail handler par handler est versionné dans **bff-handler-inventory.csv**. Le tableau ci-dessous regroupe les familles validées.

Famille	Endpoints principaux	Preuve live
Bootstrap	OAuth client credentials, getservicetoken , getserviceinfo , getregions , getguestmembertoken	HTTP 200
Catalogue	albums, album tracks, tracks, alternates, labels/libraries, featured playlists	IDs et compteurs relus
Recherche	cloudsearch , autocomplete, facets, tris, filtres, main/all versions	HTTP 200 ; contrat comparé
Assets	artwork, waveform, audio, Range	URLs utilisables ; secrets expurgés
Auth membre	login, session, persistent login, logout, refus sans cookie	parcours navigateur validés
Profil	lecture et modification réversible	valeurs relues et restaurées
Photo	presigned URL, PUT serveur, confirmation, retrait	original restauré
Favori track	addtofavourites , getfavourites , removefavouritestrack	visible dans Account après reload
Saved searches	history de recherche, add/list/remove	création, relance, suppression
Tags	add/update/list/remove	cycle complet
Relation tag-track	addtomembertags , getmembertagtracks , removetrackmembertag	fonctionnel après correctif de casse
Historique	gethistorybymembertoken	6 événements reliés aux tracks
Historique download	getdownloadhistorybymembertoken	lecture HTTP 200
Copie éditoriale	copytomemberplaylist avec CopyTracks:true	copie avec 11 tracks, puis suppression
Contact Parigo	/api/contact via Resend	HTTP 201 + accusé reçu

Les 117 tests unitaires passent. Le build Next.js de production passe sous Node 22. La suite E2E complète passe avec 135 scénarios réussis, 15 scénarios conditionnellement ignorés et 0 échec.

Fonctionnalités locales assumées

Fonction	Localité	Conclusion
Player, volume, queue, shuffle, repeat	navigateur	aucune mutation Harvest supposée
Shortlist anonyme	localStorage	correcte avant conversion
Vues liste/grille et densité	navigateur	présentation uniquement
Contact	BFF Parigo + Resend	aucun endpoint Harvest générique identifié
Éditorial compositeurs/vidéos	données Parigo	aucune entité Harvest confirmée

4. Simplifications et correctifs Parigo réalisés

Zone	Avant	Après
Search intention	interprétation et IDs en URL côté navigateur	brief envoyé au BFF ; résolution dans meta.intentResolution
Filtres Harvest	logique dupliquée	source serveur partagée
Traduction	fallback peu observable	fallback explicite ; requête appliquée exposée
Playlists	plusieurs alias non documentés	builders documentés uniquement
Copie playlist	copie parfois vide	CopyTracks:true
Shortlist	création puis ajouts séparés	transaction BFF et conservation locale sur échec
Persistence	succès fondé sur HTTP 200	relecture distante jusqu'à 30 s
Erreurs	statut amont perdu ; code 3 mal classé	statut public stable + upstreamCode expurgé
Retry	risque sur GET mutateurs	retry réservé aux lectures réellement idempotentes
Historique	dates prises sur les tracks	jointure HistoryItems.TrackID et DeliveryDate
Dates naïves	dépendance au TZ du runtime	conversion déterministe Europe/Paris ou UTCOffset
Tags	Tracks seulement	clés de tableaux tolérantes à la casse
Image profil	PUT navigateur vers URL présignée	upload multipart au BFF, limite 5 Mio
Abonnement	HTTP 200 considéré comme succès	état du profil relu et comparé

Responsabilités finales du BFF

Le BFF conserve uniquement :

1. transport, AccessKey et tokens ;
2. session chiffrée, validation et same-origin ;
3. adaptation minimale des contrats documentés ;
4. cache public, **no-store** membre, erreurs et observabilité ;
5. preuve de persistance des mutations.

Il ne doit pas devenir un second moteur de catalogue. La reconstruction des albums favoris, la suppression album track par track, les notes fondées sur un contrat incomplet et l'URL Parigo dans **Description** restent des adaptateurs temporaires à supprimer lorsqu'un contrat Harvest canonique existe.

5. Dysfonctionnements reproductibles — playlists

P1 — Création de playlist membre sans persistance

Preuve	Résultat
Endpoint	POST /addmemberplaylist/{memberToken}
Corps documenté testé	PlaylistName, PlaylistDescription
Réponse	HTTP 200, objet vide
Relecture	aucune nouvelle playlist à 0 ms → 30 s
BFF	HTTP 502 après relecture
UI shortlist	shortlist conservée ; aucun ajout de tracks déclenché
Attribution	capacité, prérequis ou comportement Harvest à confirmer

L'ancien corps Parigo (**Name**, **Description**, **IsPublic**, **PlaylistCategoryID**) et le corps documenté ont tous deux été testés. Le rapport ne présente donc plus l'ancien payload comme un problème Harvest.

P2 — Ajout et ordre de tracks

Endpoint	Corps officiel	Réponse
addtomemberplaylists	ObjectType:"Track", ObjectIDs, AddToPlaylistIDs	HTTP 200, Error.Code=4 , aucune persistance
reordermemberplaylisttracks	FromPlaylistID, ToPlaylistID, TrackIDs, un positionnement	HTTP 200, Error.Code=4
removeplaylisttracks	body absent de la documentation publique	BFF courant : HTTP 400

La documentation générale décrit le code 4 comme une erreur interne de l'opération, pas comme un manque d'authentification. Le BFF le remonte désormais en 502 non retryable.

P3 — Suggestions

POST /suggestmemberplaylisttracks/{memberToken}/{playlistId} répond HTTP 200 avec **Error.Code=3**. Ce code est documenté comme « accès refusé pour l'opération ». Le BFF renvoie désormais 403.

Question minimale : la fonctionnalité de suggestions ou de playlists collaboratives doit-elle être activée séparément sur le service, la clé, le groupe membre ou le compte ?

P4 — Partage

getinvitedmembertoken fonctionne. getsharemusicurl répond ensuite HTTP 200 avec **Error.Code=2**; l'e-mail n'est donc pas envoyé. La copie temporaire a été supprimée.

Question minimale : fournir le body canonique pour un partage de playlist **Sync/Copy** et confirmer les capacités optionnelles nécessaires.

6. Dysfonctionnements reproductibles — compte et track

Priorité	Fonction	Requête officielle/courante	Résultat observé	Question Harvest
Haute	Note privée	addtrackmembercomment , TrackID , TagName	HTTP 200, code 2, « trackid is empty »	body canonique create/update et distinction comment/tag
Haute	Abonnement	membersubscribe , e-mail + prénom/nom + Subscribe	HTTP 200 mais état inchangé	subscribe/unsubscribe partagent-ils ce contrat ?
Haute	Reset	sendpasswordresetemail , e-mail du compte	HTTP 200, code:"Failed"	prérequis et forme d'erreur attendue
Haute	Download validation	champs downloadtype , identifier , format , trims, version check	HTTP 400 sans détail utile	droits, format et exemple valide
Moyenne	Cue sheet	body courant TrackIDs et Tracks[{}ID]	HTTP 200, code 2 Incorrect Input Data	body officiel non publié
Moyenne	Photo remove	removeassignedupload	code 4, mais image réellement absente	code attendu quand l'effet réussit
Basse	Historique après écoute	URL stream réellement lue	aucune nouvelle entrée dans le contrôle immédiat	durée minimale et délai d'ingestion

Notes

La lecture **gettrackmembercomments** fonctionne. La création est bloquée ; update et delete n'ont donc pas été exécutés avec une ressource artificielle.

La documentation publique ne détaille pas le corps de **addtrackmembercomment**, **updatetrackmembercomment** ni **getcuesheet**. Aucun payload alternatif n'a été inventé.

Téléchargement

Le profil est lisible et expose les indicateurs de droit/quota, mais le payload documenté de validation reçoit HTTP 400. Pour ne pas consommer un quota sans validation, **getmusicdownload** n'a pas été appelé. Aucun téléchargement ne doit être facturé à cette campagne.

Photo

Le cycle fonctionnel est validé : sauvegarde de l'image initiale, upload neutre, confirmation, lecture, retrait, réupload de l'original et vérification. Le code 4 retourné au retrait est incohérent avec l'état final ; le BFF relit désormais le profil avant de conclure à un échec.

7. Recherche multilingue, dates et e-mails

Recherche mariage / wedding

Appel	Résultat
Harvest direct, titre mariage	0
Harvest direct, agrégé mariage	0
Harvest direct, titre wedding	160
Harvest direct, agrégé wedding	1 212
BFF littéral mariage	0
BFF intention mariage	catégorie Harvest résolue, 488 pistes

TranslateKeyword:"fr" est envoyé conformément au builder actuel, sans traduction native constatée. Parigo couvre le cas avec sa taxonomie et un fallback DeepL observable, mais la traduction, l'indexation et l'autocomplete multilingues seraient plus cohérents côté Harvest.

Questions :

- sémantique exacte de **TranslateKeyword** ;
- capacité dépendante du service, de la région, du catalogue ou d'un provider ;
- possibilité d'obtenir nativement **mariage** → **wedding**, autocomplete compris.

Date de la recherche Birthday

Harvest renvoie **CreatedDate=2026-07-29T00:24:11.477** sans offset, avec un member token portant **UTCOffset=10**, alors que l'en-tête HTTP **Date** est encore au 28 juillet UTC.

Parigo ne dépend plus du fuseau du runtime. L'entrée est affichée **28/07/2026** dans trois contextes navigateur : UTC, Europe/Paris et Australia/Sydney. La correction produit est donc faite.

Il reste utile de demander le contrat Harvest : fuseau de référence de **CreatedDate**, **CreateDate**, **LastUpdateDate**, **DeliveryDate**, et règle d'association à **UTCOffset**.

E-mails

Flux	Fournisseur actuel	Résultat
Reset password	Harvest	code:"Failed" , non reçu
Partage playlist	Harvest	bloqué avant l'envoi
Contact	Resend/Parigo	HTTP 201, accusé reçu

Aucun endpoint générique de contact n'est identifié dans la documentation Harvest. Si Harvest propose un service transactionnel, il faut connaître contrat, reply-to, webhooks, idempotence et personnalisation des templates. Sinon Resend doit rester.

8. Permissions et capacités à confirmer

Il ne faut pas inventer de scopes comme **playlists:write**. Harvest doit fournir les noms réels des rôles, scopes ou capacités.

Niveau	Besoin produit	État
Lecture publique	catalogue, recherche, assets, playlists éditoriales	fonctionnel
Lecture membre	profil, favoris, playlists, recherches, tags, commentaires, historiques	largement fonctionnel
Création membre	favoris, playlists, tracks de playlist, saved searches, tags, notes, shares	partiel
Modification	profil, abonnement, image, playlists, ordre, tags, notes	partiel
Suppression	favoris, playlists, searches, tags, notes, image	partiel
Conditionnel	download, cue sheet, e-mails, partage	à activer/confirmer

Questions de configuration

- La restriction dépend-elle de l'AccessKey, du client OAuth, du service, du groupe membre ou du compte ?
- La clé existante peut-elle être étendue ou faut-il une nouvelle clé ?
- Quelles capacités sont optionnelles ou facturées : AI search, collaborative playlists, category mapping, suggestions ?
- Existe-t-il un sandbox isolé et un membre de test ?
- Quel code HTTP et quel **Error.Code** sont attendus quand une capacité est désactivée ?
- Quelles mutations sont idempotentes et quel délai de cohérence est garanti ?

Member, management, CMS et Import

La documentation distingue les tokens membre et management ; un username membre n'est pas un credential management. Aucun fait documentaire ne permet de conclure qu'un admin Public API donne accès au CMS.

Harvest doit préciser :

- différences entre member, management user, CMS user et workspace/import user ;
- lien éventuel entre management token et CMS ;
- droits sur groupes, quotas, downloads et suggestions ;
- accès nécessaire au site versus accès éditorial/CMS.

La création ou modification du catalogue est hors périmètre actuel. Si elle devient nécessaire, elle relève de l'Import API et d'un token/session distincts.

9. Endpoints problématiques et évolutions demandées

Endpoints problématiques

Statut de preuve	Endpoint	Comportement	Limitation	Impact	Vérification demandée
Live + docs	addmemberplaylist	200 {}, absent après 30 s	création non persistée	shortlist bloquée	capacité/prérequis
Live + docs	addtomemberplaylists	code 4	aucune relation	ajout track impossible	code 4 + exemple
Live + docs	reordermemberplaylisttracks	code 4	ordre impossible	playlist incomplète	contrat/activation
Live + docs	suggestmemberplaylisttracks	code 3	accès refusé	suggestions absentes	activation exacte
Live + docs incomplètes	comments	code 2 trackid empty	body incomplet	notes impossibles	bodies create/update
Live	membersubscribe	200 sans changement	faux succès	réglage non fiable	contrat unsubscribe
Live	reset	Failed	e-mail absent	récupération bloquée	prérequis
Live + docs	download validate	HTTP 400 vide	diagnostic impossible	download bloqué	exemple + droits
Live + docs incomplètes	getcuesheet	code 2	body non publié	cue sheet bloqué	body canonique
Live + docs incomplètes	getsharemusicurl	code 2	partage interrompu	aucun e-mail	body + capacité

Évolutions Harvest

Besoin	Endpoint ou capacité	Logique à déplacer vers Harvest/BFF	Bénéfice	Priorité
Playlists membre fiables	endpoints CRUD/relations	Harvest persiste ; BFF vérifie	shortlist et Account	P0
Exemples de bodies	remove tracks, comments, cue sheet, share	contrats officiels	éviter essais non documentés	P0
Recherche multilingue	cloudsearch/autocomplete	traduction/indexation Harvest	résultats cohérents	P1
Albums favoris canoniques	add/get/remove album	éviter agrégation/suppression track par track	modèle stable	P1
Historique d'écoute	contrat d'ingestion	documenter durée/délai	Account fiable	P1
E-mail transactionnel/contact	service ou confirmation d'absence	Harvest ou Resend clairement choisi	architecture claire	P2
Templates e-mail Parigo	console/template service	branding et langues Harvest	cohérence de marque	P2

10. cURL expurgés des anomalies restantes

Les placeholders doivent être remplacés hors rapport. Aucun token réel n'est inclus.

Création playlist

```
curl -i -X POST \
  '<HARVEST_SERVICE_URL>/addmemberplaylist/<HARVEST_MEMBER_TOKEN>' \
  -H 'Content-Type: application/json' \
  --data '{"PlaylistName":"Parigo audit <RUN_ID>","PlaylistDescription":"Audit"}'
# Observé : HTTP 200, {}, aucune playlist après 30 s.
```

Ajout de tracks

```
curl -i -X POST \
  '<HARVEST_SERVICE_URL>/addtomemberplaylists/<HARVEST_MEMBER_TOKEN>' \
  -H 'Content-Type: application/json' \
  --data '{"ObjectType":"Track","ObjectIds":["<TRACK_ID>"],"AddToPlaylistIds":["<PLAYLIST_ID>"]}'
# Observé : HTTP 200, Error.Code=4, aucune persistance.
```

Suggestions

```
curl -i -X POST \
  '<HARVEST_SERVICE_URL>/suggestmemberplaylisttracks/<HARVEST_MEMBER_TOKEN>/<PLAYLIST_ID>' \
  -H 'Content-Type: application/json' \
  --data '{"Skip":0,"Limit":12,"MainOnly":true,"SeedDetermination":"Created_Desc","SeedLimit":5,"SeedMin":""}'
# Observé : HTTP 200, Error.Code=3.
```

Note privée — contrat courant

```
curl -i -X POST \
  '<HARVEST_SERVICE_URL>/addtrackmembercomment/<HARVEST_MEMBER_TOKEN>' \
  -H 'Content-Type: application/json' \
  --data '{"TrackID":"<TRACK_ID>","TagName":"Parigo audit note"}'
# Observé : HTTP 200, Error.Code=2, trackid is empty.
```

Download validation

```
curl -i -X POST \
  '<HARVEST_SERVICE_URL>/validatemusicdownloadrequest/<HARVEST_MEMBER_TOKEN>' \
  -H 'Content-Type: application/json' \
  --data '{"downloadtype":"track","identifiant":"<TRACK_ID>","format":"<FORMAT_ID>","trimstartsecs":0,"trimendsecs":0,"includeversioncheck":false}'
# Observé : HTTP 400 sans corps exploitable.
```

Les cURL pour cue sheet et partage ne sont pas présentés comme « officiels », car la documentation publique consultée ne montre pas leur body complet. Le payload et les réponses expurgées sont conservés dans la matrice de test.

11. Projet de message à Roland

Objet : Parigo × Harvest — avancement de l'intégration et capacités à confirmer

Bonjour Roland,

Merci pour ton message. Le développement a bien commencé et nous avons désormais quadrillé l'intégration de bout en bout : documentation, appels Harvest directs, BFF Parigo et parcours navigateur desktop/mobile avec un compte membre dédié.

Le socle est opérationnel : authentification de service et membre, catalogue, recherche, autocomplete, référentiels, assets, playlists éditoriales, profil, favoris tracks, recherches sauvegardées, tags, associations tag-track et historiques en lecture. Nous avons également corrigé de notre côté les payloads qui ne suivaient pas exactement la documentation, la gestion des codes fonctionnels, les contrôles de persistance, les dates/fuseaux et plusieurs mappings de réponses.

Nous avons en revanche quelques capacités membre qui restent non concluantes après test avec le contrat documenté :

- **addmemberplaylist** répond HTTP 200 mais la playlist n'apparaît pas après relecture jusqu'à 30 secondes ;
- **addtomemberplaylists** et **reordermemberplaylisttracks** renvoient **Error.Code=4** ;
- **suggestmemberplaylisttracks** renvoie **Error.Code=3** ;
- la création de commentaire/note renvoie le code 2 indiquant que **trackid** est vide, alors que **TrackID** est envoyé ;
- la validation de téléchargement répond HTTP 400 sans détail ;
- cue sheet et partage répondent code 2 avec les bodies actuellement disponibles ;
- **membersubscribe** répond positivement sans changement d'état ;
- **sendpasswordresetemail** a répondu **Failed**.

Peux-tu nous indiquer si ces écarts correspondent à des capacités à activer sur l'AccessKey, le client OAuth, le service, le groupe membre ou le compte ? Nous ne trouvons pas de noms de scopes publics et préférons utiliser les noms Harvest exacts. Dans nos échanges précédents, tu indiquais que certaines fonctions — AI search providers, collaborative playlists, category mapping — pouvaient être des options distinctes : pourrais-tu nous confirmer celles qui sont actives sur notre environnement et leur éventuel coût ?

Pour les endpoints dont le body n'est pas détaillé publiquement, pourrais-tu nous fournir un exemple canonique pour :

- **removeplaylisttracks** ;
- **addtrackmembercomment** et **updatetrackmembercomment** ;
- **getcuesheet** ;
- **getsharemusicurl** et l'envoi d'un partage.

Nous avons aussi deux sujets produit :

1. Recherche multilingue : **mariage** renvoie 0 résultat en direct, alors que **wedding** en renvoie 160 par titre et 1 212 en agrégé. Nous avons un fallback Parigo/DeepL, mais préférons une traduction/indexation native Harvest, autocomplete compris. Quelle est la sémantique exacte de **TranslateKeyword** et cette capacité doit-elle être activée ?

2. E-mails : Harvest propose déjà des flux de vérification, reset et partage. Existe-t-il également un endpoint transactionnel générique utilisable pour notre formulaire de contact ? Sinon nous conserverons Resend. Nous souhaitons aussi connaître le processus pour personnaliser tous les templates Harvest à la charte Parigo : HTML/texte, logo, couleurs, typographies, langues, sender domain, reply-to, staging/prévisualisation, SPF/DKIM/DMARC et coût éventuel.

Enfin, peux-tu nous préciser la distinction entre compte membre, management user, utilisateur CMS et workspace/import user ? Notre compte de test n'est pas admin et nous ne voulons pas assimiler accès management, CMS et Public API sans confirmation.

Nous pouvons t'envoyer les cURL expurgés et les réponses reproductibles pour chacun de ces cas. L'objectif est de circonscrire les demandes de support aux seules capacités ou zones de documentation qui restent ambiguës ; tous les écarts corrigearables côté Parigo ont déjà été traités.

Merci,

Yoann

12. État final, limites et artefacts

État du compte

- aucune suppression de compte appelée ;
- photo originale restaurée ;
- profil et abonnement restaurés ;
- favoris, tags, recherches et playlists temporaires nettoyés ou non créés ;
- aucune ressource non identifiable supprimée ;
- aucun téléchargement consommé ;
- aucun mot de passe modifié.

Limites explicites

- réception du reset et du partage impossible, car Harvest n'a pas atteint une réponse d'envoi réussie ;
- update/delete d'une note non testables sans note créée ;
- téléchargement réel non déclenché après l'échec de validation ;
- CMS/management/import non exécutés sans credentials dédiés ;
- aucune politique de fuseau Harvest déduite de sa localisation australienne.

Artefacts versionnés

- **docs/harvest/audit-integration-2026-07-28.md** — présent rapport ;

- **docs/harvest/runtime-route-matrix.csv** — preuve action par action ;
- **docs/harvest/endpoint-classification.csv** — 255 endpoints classés ;
- **docs/harvest/bff-handler-inventory.csv** — 75 handlers cartographiés ;
- **docs/harvest/last-audit-run.json** — synthèse expurgée ;
- **output/pdf/audit-integration-harvest-2026-07-28.pdf** — version Roland.

Conclusion

Le BFF simplifié est nécessaire et proportionné. Le site peut avancer sur les lectures publiques, l'authentification, les favoris, les recherches sauvegardées, les tags et le profil. Le prochain bloc fonctionnel dépend de la clarification ou de l'activation des écritures playlist, comments, downloads, cue sheets et shares.